R4.Real.10 Complément web

R4.Real.10 Complément web

Publication date 2026

Table of Contents

1. Introduction 1
    1. Programme national 1
    2. Adaptation locale 1
    3. Mise en oeuvre 2
2. DOM 3
    1. Rappels de cours 3
        1.1. JavaScript les bases du langage 3
            1.1.1. Comment charger un script JS 3
            1.1.2. Les variables et les types 4
            1.1.3. Les tableaux, les objets littéraux et le JavaScript Object Notation (JSON) 5
            1.1.4. Créer une classe, instancier un objet 6
            1.1.5. Les fonctions fléchées 7
        1.2. La manipulation de la DOM 8
            1.2.1. L'objet window 8
            1.2.2. L'objet document 8
            1.2.3. Manipuler la DOM 10
        1.3. La gestion des événements 12
            1.3.1. Utilisation 12
            1.3.2. L'objet Event 13
        1.4. Littéraux de gabarits 14
        1.5. Opérateur de décomposition, map, reduce et filter 14
            1.5.1. L'opérateur de décomposition 14
            1.5.2. map, reduce, filter 14
        1.6. Les modules en JS 15
        1.7. Le mode strict 15
        1.8. Les appels asynchrones 16
            1.8.1. Une introduction au promesses 16
            1.8.2. Une introduction aux fonction asynchrones 16
            1.8.3. L'API fetch 16
    2. Exercices 17
        2.1. Exercices 17
            2.1.1. Exercice 1 17
            2.1.2. Exercice 2 17
            2.1.3. Exercice 3 18
            2.1.4. Exercice 4 18
3. React 21
    1. Composants 21
        1.1. Cours 21
            1.1.1. Un premier composant 21
            1.1.2. Ajout de JSX 22
            1.1.3. Composons/décomposons nos composants 23
            1.1.4. État local et cycle de vie 26
            1.1.5. Les événements 29
            1.1.6. Les formulaires 30
            1.1.7. Les affichages conditionnels 31
            1.1.8. Les listes 34
            1.1.9. Faire remonter un état 36
        1.2. Exercices 38
            1.2.1. Le tutoriel officiel 38
            1.2.2. Exercice 1 38
            1.2.3. Exercice 2 38
            1.2.4. Exercice 3 38
    2. Composants communicants et routage (non évalué en cours) 39
        2.1. Un CLI 39
        2.2. Le routage 40

2.3. Propager les props 40
2.4. Redux 41

List of Figures

2.1. DOM 9
2.2. DOM héritage 9
2.3. L'image du bas en remplacée par celle du haut 17
2.4. Modale visible 18
2.5. sessionStorage 18
2.6. Page d'accueil (DOM) 19
2.7. film (DOM) 19
3.1. Cycle de vie d'un composant React 26

List of Tables

1.1. Planning 1
2.1. Événements standards 12
2.2. Événements de la balise `form` 13

Chapter 1. Introduction

1. Programme national

L’objectif de cette ressource est de compléter les techniques de développement web pour arriver à des capacités de développement full-stack. Les avoirs faire de référence étudiés sont :

- Programmation web (par ex. : programmation côté client...)
- Mise en oeuvre de l’ergonomie
- Mise en œuvre de l’accessibilité

2. Adaptation locale

Nous allons utiliser l'API de "the movie db" pour offrir un site sur les films : https://www.themoviedb.org/documentation/api. Vous pouvez déjà créer un compte pour obtenir votre clef d'API dans les paramètres de votre compte. Voici quelques URL intéressantes que vous pouvez tester :

Films

- https://api.themoviedb.org/3/movie/${id}?api_key=${key}&language=en-US
- https://api.themoviedb.org/3/movie/${id}/credits?api_key=${key}
- https://api.themoviedb.org/3/movie/${id}/recommendations?api_key=${key}&language=en-US&page=${page}
- https://api.themoviedb.org/3/movie/popular?api_key=${key}&language=en-US&page=${page}

Acteurs

- https://api.themoviedb.org/3/person/${actorId}?api_key=${api_key}&language=en-US
- https://api.themoviedb.org/3/person/${actorId}/movie_credits?api_key=${api_key}&language=en-US

Recherche

- https://api.themoviedb.org/3/search/movie?api_key${key}&language=en-US&query=${term}&page=${page}&include_adult=false

Images

- https://image.tmdb.org/t/p/w1280${backdrop_path}

La progression pédagogique théorique sera :

Table 1.1. Planning

Semaine	CM	TD	Évaluation
S10	1h20 DOM	4h DOM	
S11	1h20 React Intro	4h DOM	
S12		4h React (Non alternants)	
S13	1H20 React	4h React	
S14	1H20 React Native	1h20 React	2h40 CC

3. Mise en oeuvre

Il vous faudra un serveur HTTP, vous pouvez opter pour le serveur léger de votre choix qui supporte les modules, comme par exemple http-server (`https://www.npmjs.com/package/http-server`) :

```
npx http-server
```

N'oubliez pas que vous ne pouvez accéder à localhost, il vous faut passer par l'adresse ip (**ip a**).

Chapter 2. DOM

1. Rappels de cours

Vous avez déjà vu en grande partie, les éléments de ce cours dans le cours R4.01. Les spécificités portent sur la DOM.

1.1. JavaScript les bases du langage

Le JavaScript est un langage orienté objet, faiblement typé, interprété et exécuté principalement coté client. Ce langage date de 1996 et en est à la version 1.8.15 (Implémentation basée sur les spécifications de la 5e édition du standard ECMA-262.), il est supporté nativement par l'ensemble des navigateurs et permet de rendre vos pages plus dynamiques. Rappelez vous en CSS, vous ne disposiez des pseudo classes (:hover, :active, :focus, ...) et des pseudo éléments (::after, ::before) qui permettent de modifier l'HTML ; avec cette approche comment, par exemple réaliser un drag and drop, JavaScript et la DOM le permettent.

Dans suite de la partie cours, nous allons introduire sommairement le langage JavaScript, des compléments seront donnés au fur et à mesure des exercices. La documentation recommandée et disponible le jour de l'examen, se trouve ici : `https://developer.mozilla.org/fr/docs/Web/JavaScript`.

1.1.1. Comment charger un script JS

Vous pouvez charger un script JS de trois manière :

Dans une balise

par exemple, le click sur un bouton déclenche un message dans la console.

```
<button onclick="console.log('tu as cliqué')">un bouton</button>
```

Dans l'entête

placé dans l'entête le script est chargé avant la construction de la page, nous verrons plus tard qu'il est possible de le mettre en attente de la fin du chargement.

```
<script type="text/javascript" src="./ressources/exemple01.js"></script>
```

Dans la page

Il est également possible d'utiliser la balise script dans le `<body>`, le script disposera des éléments créés précédemment.

```
<script type="text/javascript">
console.log("je suis la");
</script>
```

L'exemple suivant vas produire dans la console (`console.log()`)les affichages suivants :

```
<!-- exemple01.html -->
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Exemple 01</title>
<script type="text/javascript" src="./ressources/js/exemple01.js"></script>
</head>
<body>
<button onclick="a = a +1; console.log(a,' sur un click')">un bouton</button>
<script type="text/javascript">
 a = a + 1;
 console.log(a, " dans la balise script");
</script>
</body>
```

```
</html>
```

```
// ./ressources/js/exemple01.js
let a=1; //décalaration de a et affectation de 1
console.log(a, "dans exemple01.js");
```

```
1 "dans exemple01.js"         exemple01.html:12
2 " dans la balise script"    exemple01.html:9
3 " sur un click"             exemple01.html:9 4 " sur un click"
```

Important

L'ordre du chargement des scripts a son importance, ce qui est créé dans un script est visible dans ceux qui le suivent.

1.1.2. Les variables et les types

Le mot clef `let` (ou `const` pour les constantes) permet de définir une variable de portée locale au block qui la contient. Sans le mot clef `let`, `var` ou `const` une affectation crée une variable de portée globale. Le langage n'a pas de typage explicite mais à chaque instant une variable définie peut avoir un type :

Booléén

les 2 seules valeurs possibles sont `true` ou `false`, les opérateurs logiques sont `||` (ou), `&&` (et), `!` (négation);

Null

la seule valeur possible est `null`;

indéfini

une variable non définie (non initialisée) aura la valeur `unedefined`;

nombre

les nombres en virgule flottante, il n'existe pas de type spécifique pour les entiers, les valeurs symbolique suivante existent `+Infinity` (infini positif), `-Infinity` (infini négatif), et `NaN` (Not A Numérical Value);

String

les chaînes de caractère

Symbole

un symbole est une valeur primitive unique et immuable pouvant être utilisée comme clé pour propriété d'un objet;

Object

les objets en JavaScript, pour information les tableaux sont des objets, les objets peuvent être construit avec un `new` ou sous forme littérale, le modèle objet repose sur *prototype*, une approche différente des *classes* que vous avez vu en PHP.

L'exemple suivant illustre la possibilité de changer de type à l'exécution, l'opérateur `typeof` permet de connaître le type.

```
let a = 2; //déclaration de a comme étant un entier
 console.log(typeof a); //number
 let b= a +"4"; //Le + est la concaténation
 console.log(typeof b); //String
 let c=  parseFloat(b) //parseFloat permet d'obtenir un nombre
 console.log(typeof c); //Number
 let d = ["Un", "Deux", "Trois"]; //un tableau indicé de 3 éléments
 console.log(typeof d); //Object
 let e = {nom:"Lestideau", prenom:"vincent"};// Un objet litteral avec 2 attributs
 console.log(typeof e); //Object
```

Il vous faut proscrire l'usage de var, var est similaire à let, mais avec une portée de fonction (là ou let a une portée de bloc), dont si vous définissez en dehors d'une fonction, la portée sera celle du contexte global.

Pour finir, JS possède un mécanisme de hoisting, les variables sont déclarées avant toute exécution :

```
function fn() {
  console.log(foo); // undefined (au lieu de ReferenceError)
  var foo = "bar";
}

function fn() {
  console.log(foo);
  // ReferenceError, on est dans la Temprale Dead Zone pour la variable foo
  let foo = "bar";
}
```

1.1.3. Les tableaux, les objets littéraux et le JavaScript Object Notation (JSON)

L'enseignement du modèle objet de JavaScript sera vu dans le module M4105, nous allons simplement voir les objets littéraux. Cette approche est pour le moment suffisante.

JSON est format des objets littéraux, il peut-être transformé en chaîne de caractère avec JSON.stringify() et une chaîne peut-être transformée en JSON avec JSON.parse(), ces 2 opérateurs seront utiles lorsque nous souhaiterons envoyer et recevoir des données à travers le réseau.

Un objet littéral est défini comme suit l

```
et nomVariable = {nomAttribut1 : valeur1, nomAttribut2: valeur2}
```

, par exemple

```
let personne1 = {nom: "Durand", prenom:"Paul",
parle : function(){console.log("Je suis :" + this.nom+" "+this.prenom)}};.
```

L'accès aux propriétés de l'objet se fait en utilisant . , l'instance courante est représentée par `this`.

Les tableaux en JavaScript sont indexés à partir de 0, le code suivant illustre plusieurs moyens de créer des tableaux:

```
let myArray1 = new Array('Sébastien', 'Laurence', 'Ludovic', 'Pauline', 'Guillaume');
let myArray2 = ['Sébastien', 'Laurence', 'Ludovic', 'Pauline', 'Guillaume'];
let myArray3 = new Array();
myArray3.push('Sébastien');myArray3.push('Laurence'); //push ajoute à la fin du tableau
```

`.length` permet de connaître la taille tu tableau, `unshift(elt)` permet d'ajouter au début du tableau, `push(elt)` à la fin du tableau, `shift()` retire le premier élément et pop() retire le dernier élément.

L'exemple qui suit nous servira de lien dans les exercices qui vont suivre, c'est un exemple de JSON fourni par l'API de flickr avec l'URL `https://www.flickr.com/services/rest/?method=flickr.photos.search&api_key=649a72bf0e3c6917502db713dce1bb79&tags=furet&format=json&nojsoncallback=1&api_sig=f26a10c36a2d401c0b6036aab67f21c9` . L'API flickr est un service REST (Representational State Transfer) qui nous permet de gérer des images, ici nous avons demandé les 4 derniers furets (`method=flickr.photos.search`, `tags=furet`, `per_page=4`).

```
{ "photos": { "page": 1, "pages": "1017", "perpage": 4, "total": "4065",
    "photo": [
      { "id": "33276227384", "owner": "95362858@N03",
        "secret": "eea1701366", "server": "2928", "farm": 3,
        "title": "Nice Art + Paddy_4739 avenue du Général Jean Simon Paris 13",
        "ispublic": 1, "isfriend": 0, "isfamily": 0 },
      { "id": "33165599206", "owner": "36522981@N00",
        "secret": "a4b92d51e3", "server": "3776", "farm": 4,
        "title": "spring in the air", "ispublic": 1, "isfriend": 0, "isfamily": 0 },
      { "id": "32987754821", "owner": "144782736@N08",
        "secret": "f99d8bb508", "server": "2825", "farm": 3,
        "title": "20170225214115_IMG_0737", "ispublic": 1, "isfriend": 0, "isfamily": 0 },
      { "id": "33072968806", "owner": "144782736@N08",
        "secret": "e5f8eeec51", "server": "772", "farm": 1,
```

```
        "title": "20170225214031_IMG_0733", "ispublic": 1, "isfriend": 0, "isfamily": 0 }
    ] }, "stat": "ok" }
```

Le code suivant permet d'exploiter le flux JSON :

```
 const s = '{ "photos": { "page": 1, "pages": "1017", "perpage": 4, "total": "4065",'+
      '"photo": ['+
          '{ "id": "33276227384", "owner": "95362858@N03", "secret": "eea1701366",'+
            '"server": "2928", "farm": 3, '+
            '"title": "Nice Art + Paddy_4739 avenue du Général Jean Simon Paris 13",'+
            '"ispublic": 1, "isfriend": 0, "isfamily": 0 },'+
          '{ "id": "33165599206", "owner": "36522981@N00", "secret": "a4b92d51e3",'+
             '"server": "3776", "farm": 4, "title": "spring in the air",'+
             '"ispublic": 1, "isfriend": 0, "isfamily": 0 },'+
          '{ "id": "32987754821", "owner": "144782736@N08", "secret": "f99d8bb508",'+
             '"server": "2825", "farm": 3, "title": "20170225214115_IMG_0737",'+
             '"ispublic": 1, "isfriend": 0, "isfamily": 0 },'+
          '{ "id": "33072968806", "owner": "144782736@N08", "secret": "e5f8eeec51",'+
             '"server": "772", "farm": 1, "title": "20170225214031_IMG_0733",'+
             '"ispublic": 1, "isfriend": 0, "isfamily": 0 }'+
        '] }, "stat": "ok" }';
  //s est ici une chaîne
  const myJSON = JSON.parse(s);
     // myJSON est un objet composé de 2 attributs :
     // photos un objet et stat une chaîne de caractère qui vaut "ok"
     // l'objet photos contiend 5 attributs :
     // photos.page un nombre de valeur 1
     // photos.pages une chaîne de valeur 1017
     // photos.perpage un entier de valeur 4
     // photos.total une chaîne de valeur 4065
     // photos.photo un tableau de 5 objets
     // photos.photo[0] est le premier élément du tableau et est un objet avec 9 attributs
     // photos.photo[0].id est une chaîne de valeur 33276227384
     // ....
  console.log(myJSON);
```

Dans la console, nous obtenons :

```
Object {photos: Object, stat: "ok"}
  photos : Object
    page : 1
    pages: "1017"
    perpage : 4
    photo : Array(4)
      0 : Object
        farm : 3
        id : "33276227384"
        isfamily : 0
        isfriend : 0
        ispublic : 1
        owner  : "95362858@N03"
        secret : "eea1701366"
        server : "2928"
        title : "Nice Art + Paddy_4739 avenue du Général Jean Simon Paris 13"

...
```

1.1.4. Créer une classe, instancier un objet

JavaScript est un langage objet basé sur des prototypes plus que sur des classes, nous allons ici introduire sommairement les classes. Il faut savoir qu'en JS à l'exécution, il est possible d'ajouter et retirer des attributs à un objet à l'exécution, voir à l'ensemble des objets si l'on passe par le prototype. Enfin comme une variable peut changer de type à l'exécution, par exemple un entier peut devenir une fonction. Voici un petit exemple avec un usage dans l'esprit de Java.

```
  class Animal {
        #nom; //atribut privé
        //Le constructeur des valeurs par défaut sont possibles
        //pour les paramètres
```

```
        constructor(nom) {
            this.#nom = nom || "Truc";

        }
        //Une note
        parle() {
            console.log(`${this.nom} fait du bruit.`);
        }

        //un accesseur
        get nom(){
            return this.#nom.toUpperCase();
        }

        //Une toString
        toString() {
            return `Animal : ${this.nom}`;
        }
    }

    //heritage
    class Chien extends Animal {
        constructor(nom="JoJo") {
            super(nom); // appelle le constructeur parent avec le paramètre
        }
        parle() {
            console.log(`${this.nom} aboie.`);
        }
    }

    //test
    let animal = new Animal();
    const nom = animal.nom;
    console.log(animal, nom);
    animal.parle();
    animal = new Chien();
    animal.parle();
```

Les affichages produits sont les suivants :

```
Animal {#nom: "Truc"} "TRUC"
TRUC fait du bruit.
JOJO aboie.
```

1.1.5. Les fonctions fléchées

En JS les fonctions sont des constructeurs auto-déclarant, par exemple :

```
  const Animal = function(nom="JoJo"){
        this.nom = nom;
    }
    const a = new Animal();
    console.log(a);
```

Ainsi, toutes les fonctions "classiques" disposent d'un `this` propre et ne peuvent plus accéder au `this` englobant. Les fonctions fléchés sont des fonctions sans `this` propre. La syntaxe de base est

```
([param] [, param]) => {
   instructions
}

(param1, param2, …, param2) => expression
// équivalent à
(param1, param2, …, param2) => {
  return expression;
}

// Parenthèses non nécessaires quand il n'y a qu'un seul argument
param => expression
```

```
// Une fonction sans paramètre peut s'écrire avec un couple
// de parenthèses
() => {
  instructions
}
```

Voici un exemple impossible et sa solution avec des fonctions fléchées.

```
function Personne () {
  // Le constructeur Personne() définit `this` comme lui-même.
  this.age = 0;

  setInterval(function grandir () {
    // En mode non strict, la fonction grandir() définit `this`
    // comme l'objet global et pas comme le `this` defini
    // par le constructeur Personne().
    this.age++;
  }, 1000);
}

const p = new Personne();

//Solution
function Personne () {
  this.age = 0;

  setInterval(() => {
    this.age++;
    // |this| fait bien référence à l'objet personne
  }, 1000);
}

const p = new Personne();
```

1.2. La manipulation de la DOM

La DOM (Document Object Model) est la représentation sous la forme d'un arbre d'objet de votre navigateur.

1.2.1. L'objet window

L'objet window représente la fenêtre du navigateur. C'est à partir de cet objet que le JavaScript est exécuté. A titre d'exemple vous pouvez tester dans la console de votre navigateur le code suivant :

```
window.print();
```

L'objet `window` permet également d'accéder à vos cookies, à l'historique de votre navigation, ...

Les méthodes comme `prompt` ou `alert` sont à proscrire car bloquées par les navigateurs modernes.

1.2.2. L'objet document

L'objet document nous intéresse plus particulièrement car il représente la page web, nous allons pouvoir manipuler à volonté depuis JavaScript notre page, pour la rendre plus interactive. La page web est vue comme un arbre d'objets, par exemple :

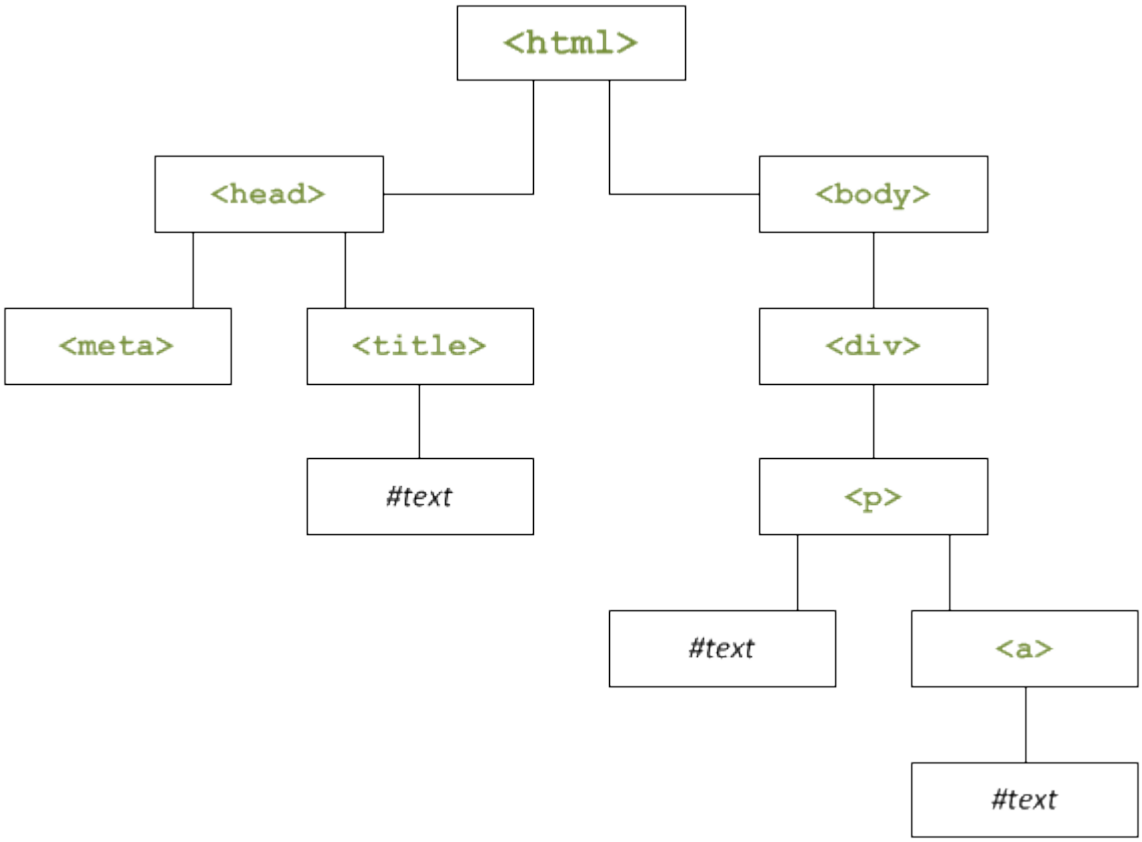
```
<!doctype html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Le titre de la page</title>
</head>

<body>
    <div>
        <p>Un peu de texte <a>et un lien</a></p>
    </div>
```

```
</body>
</html>
```

correspond à l'arbre

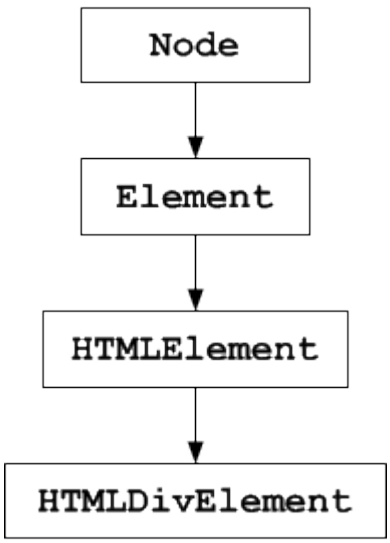
Figure 2.1. DOM



C'est arbre est composé de noeuds éléments les balises et de noeuds texte, les éléments possèdent comme tous les objets des attributs. Les noeuds peuvent avoir des enfants par exemple `html` à deux enfants `head` et `body`.

Il existe une notion d'héritage, chaque élément dispose des propriétés de son père (attribut et méthodes) plus les siennes.

Figure 2.2. DOM héritage



Ainsi, par exemple une `div` est un élément HTML spécialisé.

1.2.3. Manipuler la DOM

La liste des méthodes qui suit permet la manipulation de la page mais ne se veut pas exhaustive.

1.2.3.1. Accéder aux éléments

Les anciennes méthodes étaient `getElementById()`, `getElementsByTagName()` et `getElementsByName()` qui permettaient respectivement de récupérer un élément à partir de son identifiant, des éléments à partir du nom de la balise, des éléments à partir de l'attribut `name` (formulaire et map).

Nous utiliserons querySelector() et querySelectorAll() qui utilisent les sélecteurs CSS, querySelector renvoie le premier élément et querySelectorAll() un tableau d'éléments.

```
<body>
 <p> paragraphe 1 </p>
 <p> paragraphe 2 </p>
 <p> paragraphe 3</p>
 <script type="text/javascript">
  let paragraphes = document.querySelectorAll('p');
  console.log(paragraphes);
  //[p, p, p]
  console.log(paragraphes.length);
  //3
  let paragraphe = document.querySelector('p');
  console.log(paragraphe);
  //<p> paragraphe 1 </p>
 </script>
</body>
```

1.2.3.2. Éditer les éléments

Nous allons illustrer les méthode getAttribute() (obtenir la valeur d'un attribut), setAttibute() (modifier la valeur d'un attribut)*Le mot class est résérvé en JavaScript pour manipuler une classe il faut utiliser className*. Le code suivant trouve les élément de classe `colored` leur applique la classe `blue` et cache l'élément de classe `hidden`.

```
<!DOCTYPE html>
<html land="fr>
<head>
<meta charset="UTF-8">
<title>Exemple 06</title>
<style>
.blue {
 color: blue;
}
</style>
</head>
<body>

 <p class="colored">paragraphe 1</p>
 <p class="hidden">paragraphe 2</p>
 <p class="colored">paragraphe 3</p>
 <script type="text/javascript">
     //on récupére tous les paragraphes de class colored
  const paragraphes = document.querySelectorAll(".colored");
  //une boucle forEach permet de parcourir un tableau
  // tableau.forEcah(function(elementCourant){traitement})
  // on parcours les paragraphes récupérés
  paragraphes.forEach(function(paragraphe) {
   //pour chaque paragraphe recupéré on change la classe
   paragraphe.className = 'blue';
  });

  //on récupére le premier pragraphe
  let paragraphe = document.querySelector('.hidden');
  paragraphe.setAttribute("style","visibility : hidden");
```

```
 </script>
</body>
</html>
```

Notre code affiche en bleu :

```
paragraphe 1


paragraphe 3
```

Pour modifier le contenu d'un noeud texte, vous pouvez utiliser l'attribut `innerHTML` :

```
const text = 'hello';
//récupération de la div
const div = document.querySelector("div#display");
//modification du contenu de la div avec la saisie
div.innerHTML = text;
```

1.2.3.3. Naviguer dans les éléments

Il est parfois utile de trouver ces ancêtres ou ses enfants, le méthodes suivantes le permettent.

L'attribut `parentNode` permet de récupérer l'élément parent.

`nodeType` et `nodeName` permettent de connaître le type et le nom de l'objet courant.

`childNodes` récupère un tableau contenant les enfants, `firstChild` le premier et `lastChild` le dernier.

nextSlibling et previonSlibling permettent d'accéder au noeud précédant et au noeud suivant.

Enfin `nodeValue` permet de récupérer le contenu d'un noeud texte.

Le programme suivant transforme le contenu des paragraphes en majuscule :

```
<div>
 <p> paragraphe 1 </p>
 <p> paragraphe 2 </p>
 <p> paragraphe 3 </p>
 </div>
 <script type="text/javascript">
   //on recupère la div
   const div = document.querySelector("div");
   //on récupère tous les enfants de la div
   //attention les espaces entre les paragraphes sont des noeuds text
   const paragraphes = div.childNodes;
   //on parcoure les enfants
   paragraphes.forEach(function(paragraphOrText){
    //Si l'élément courant est un noeud élément
    if (paragraphOrText.nodeType == 1) {
     //on récupére son premier enfant, ici un noeud text
     //et on passe son contenu en majuscule toUpperCase()
     paragraphOrText.firstChild.nodeValue =
        paragraphOrText.firstChild.nodeValue.toUpperCase();
    }
   });
 </script>
```

1.2.3.4. Créer et supprimer des éléments

L'objet document contient la méthode `createElement()` qui permet de créer un élément, la méthode `createTextNode()` qui permet de créer un élément et la méthode `appendChild()` qui permet d'ajouter un enfant.

Le code suivant permet de créer un ensemble d'images cliquables à partir d'un tableau d'URL :

```
<div id="display">
```

```
</div>
<script type="text/javascript">
  //le tableau d'url
  let urls = ['https://farm2.staticflickr.com/2928/33276227384_eea1701366.jpg',
   'https://farm4.staticflickr.com/3776/33165599206_a4b92d51e3.jpg',
   'https://farm3.staticflickr.com/2838/34076072516_5df307e147.jpg',
   'https://farm3.staticflickr.com/2923/33731983300_1b5ec63ed1.jpg'];
  //on récupérer la div
  let div = document.querySelector("div#display");
  //on parcours le tableau d'urls
  urls.forEach(function(url){
   // on créé une ancre
   let a = document.createElement('a');
   // on modifie le href de l'ancre
   a.setAttribute("href",url);
   //on créé une img
   let img = document.createElement('img');
   // on modifie le src de l'image
   img.setAttribute("src",url);
   // on ajoute l'image à l'ancre
   a.appendChild(img);
   // on ajoute l'ancre à la div
   div.appendChild(a);
  });
</script>
```

D'autres modifications des éléments sont possibles comme : `removeChild()` suppression d'un enfant, `cloneNode()` clonage d'un élément et `replaceChild()` remplacement d'un élément par un autre, `insertBefore()`.

1.3. La gestion des événements

Les événements permettent de déclencher une fonction lorsqu'une action c'est produite, par exemple, lors d'un clique sur un bouton, ou la fin du chargement de la page.

1.3.1. Utilisation

Le tableau suivant donne une liste non exhaustive des événements[1] :

Table 2.1. Événements standards

Nom de l'événement	Action pour le déclencher
click	Cliquer (appuyer puis relâcher) sur l'élément
dblclick	Double-cliquer sur l'élément
mouseover	Faire entrer le curseur sur l'élément
mouseout	Faire sortir le curseur de l'élément
mousedown	Appuyer (sans relâcher) sur le bouton gauche de la souris sur l'élément
mouseup	Relâcher le bouton gauche de la souris sur l'élément
mousemove	Faire déplacer le curseur sur l'élément
keydown	Appuyer (sans relâcher) sur une touche de clavier sur l'élément
keyup	Relâcher une touche de clavier sur l'élément
keypress	Frapper (appuyer puis relâcher) une touche de clavier sur l'élément
focus	« Cibler » l'élément
blur	Annuler le « ciblage » de l'élément

[1] source: https://developer.mozilla.org/fr/docs/Web/Events.

Nom de l'événement	Action pour le déclencher
change	Changer la valeur d'un élément spécifique aux formulaires (input,checkbox, etc.)
input	Taper un caractère dans un champ de texte (son support n'est pas complet sur tous les navigateurs)
select	Sélectionner le contenu d'un champ de texte (input,textarea, etc.)

Il existe aussi des événements spécifiques aux balises `form` :

Table 2.2. Événements de la balise `form`

Nom de l'événement	Action pour le déclencher
submit	Envoyer le formulaire
reset	Réinitialiser le formulaire

Pour utiliser les événements voici trois méthodes, la dernière étant préconisée.

Il est possible d'utiliser l'événement directement dans la balise*le return false permet de bloquer l'événement* :

```
<a href="#" onclick="console.log('Vous avez cliqué !'); return false;">Cliquez-moi !</a>
```

Dans le JavaScript en utilisant le nom de l'événement :

```
<span id="clickme">Cliquez-moi !</span>

<script>

    const element = document.getElementById('clickme');

    element.onclick = function() {
        console.log("Vous m'avez cliqué !");
    };

</script>
```

En utilisant, l'approche contemporaine (DOM-2) :

```
<span id="clickme">Cliquez-moi !</span>

<script>
    const element = document.querySelector('#clickme');

    element.addEventListener('click', function() {
        console.log("Vous m'avez cliqué !");
    });
</script>
```

1.3.2. L'objet Event

L'objet évent est reçu en paramètre de la fonction traitant l'événement :

```
element.addEventListener('click', function(e) {
    // L'argument « e » va récupérer une référence vers l'objet « Event »
    console.log(e.type); // Ceci affiche le type de l'événement (click, mouseover, etc.)
});
```

Il permet de récupérer des informations liées à l'événement comme le composant source, la position du curseur, ...

Le code suivant peut vous être utile, il permet d'attendre que le contenu de la page soit disponible :

```
window.addEventListener("DOMContentLoaded",()=>{//TODO})
```

1.4. Littéraux de gabarits

Les littéraux de gabarits sont des littéraux de chaînes de caractères permettant d'intégrer des expressions. Avec eux, on peut utiliser des chaînes de caractères multi-lignes et des fonctionnalités d'interpolation. `${expr}` permet dévaluer une expression.Pour utiliser des accents graves dans un gabarit, on les échappera avec une barre oblique inverse (\) .

```
const a=22;
const s = "La valeur de a est ${a}";
console.log(s); // La valeur de a est 22
```

1.5. Opérateur de décomposition, map, reduce et filter

La programmation fonctionnelle est un paradigme de programmation de type déclaratif qui considère le calcul en tant qu'évaluation de fonctions mathématiques. Comme le changement d'état et la mutation des données ne peuvent pas être représentés par des évaluations de fonctions, la programmation fonctionnelle ne les admet pas, au contraire elle met en avant l'application des fonctions, contrairement au modèle de programmation impérative qui met en avant les changements d'état*wikipédia*.

Les fonctions fléchées que nous utilisons depuis le début de se cours répondent à ce paradigme, nous allons voir quelques méthodes de l'objet Array.

1.5.1. L'opérateur de décomposition

L'opérateur de décomposition permet de développer une expression là où plusieurs argument (pour les appels de fonction) ou plusieurs éléments (pour les littéraux de tableaux) sont attendus.

```
"use strict";
let etudiants=[
    {nom : "Durand", note : 20},
    {nom : "Dupond", note : 10},
    {nom : "Boob",  note : 0},
];

console.log(...etudiants);
//  {nom: 'Durand', note: 20 } { nom: 'Dupond', note: 10 } { nom: 'Boob', note: 0 }

etudiants = [
    {nom : "Alice", note : 16}, ...etudiants, {nom: "Jean", note:4}
    ];
console.log(etudiants);
//[
//   { nom: 'Alice', note: 16 },
//   { nom: 'Durand', note: 20 },
//   { nom: 'Dupond', note: 10 },
//   { nom: 'Boob', note: 0 },
//   { nom: 'Jean', note: 4 }
// ]

const f = (a,b,c)=> a;
console.log(f(...etudiants));
//{ nom: 'Alice', note: 16 }
```

1.5.2. map, reduce, filter

map transforme un Array, reduce le converti en un résultat et filter filtre.

```
const notes = etudiants.map(etudiant => etudiant.note);
console.log(notes);
//[ 16, 20, 10, 0, 4 ]

let somme = 0;
somme = notes.reduce((somme, note)=> somme+note);
const moyenne = somme / notes.length;
console.log(somme, moyenne);
```

```
//50 10

const bonsEtudiants = etudiants.filter(etudiant => etudiant.note > moyenne);
console.log(bonsEtudiants);
//[ { nom: 'Alice', note: 16 }, { nom: 'Durand', note: 20 } ]
//...
//lire l'API avant de coder
```

1.6. Les modules en JS

Il est possible de structurer votre application en modules, ES6 vient avec sont propre système, mais d'autres systèmes existent comme CommonJS en nodeJS. Nous allons ici, illustrer les modules natif. Un module se charge avec la balise

```
<script type="module" src="nomModule.mjs"></script>
ou
<script type="module">
  //code du script utilisant les modules ici
</script>
```

Une fois les modules chargés, `import` et `export` permettent de les utiliser, vous trouvez une documentation ici `https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Modules`. Il faut également savoir que import peut être utilisé comme une promesse pour importer dynamiquement les modules.

```
<script type="module" src="Velo.mjs"></script>
<script type="module" src="testVelo.mjs"></script>
```

```
//Velo.mjs
class Vehicule {
    type;
    constructor(type) {
        this.type = type;
    }

    toString() {
        return  'Le type est '+this.type;
    }
}

class Velo extends Vehicule {
    marque;
    constructor(marque, type) {
        super(type);
        this.marque = marque;
    }
    toString() {
        return  super.toString()+" la marque est "+this.marque;
    }
}

const velo = new Velo("m","c");

export {Velo, velo};
```

```
import {velo, Velo} from './Velo.mjs';

console.log(velo); //OK
//const v = new Vehicule("a");
//Vehicule n'est pas lisible
const v2 = new Velo("m","t"); //OK
```

1.7. Le mode strict

Le mode strict permet de choisir une variante restrictive de JavaScript. Le mode strict n'est pas seulement un sous-ensemble de JavaScript : il possède intentionnellement des sémantiques différentes du code normal. Il s'invoque avec :

```
"use strict";
```

Je vous conseille de l'utiliser principalement car :

1. Il convertit les fautes en erreurs
2. Sécurise le code
3. Augmente les performances

1.8. Les appels asynchrones

1.8.1. Une introduction au promesses

Une promesse est un objet (Promise) qui représente la complétion ou l'échec d'une opération asynchrone.

```
const truc = function (){
    return new Promise((resolve, reject) => {
        if (Math.random() > .5)
            resolve("OK");
        else
            reject("KO");
    })
}

truc().then(resp => console.log(resp)).catch(resp => console.log(resp));
//OK dans then, KO dans catch
```

Le code de `then` d'une promesse est exécuté en cas de succès (`resolve`) et cas d'échec (`reject`) le code de `catch` est exécuté. Les callback sont rattachées à `then` et `catch`. Les promesses peuvent être composées en séquence ou en parallèle, vous trouverez plus d'information ici : `https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Utiliser_les_promesses`.

Pour ceux qui souhaitent aller plus loin, vous trouverez une introduction à la concurrence : `http://blog.js-republic.com/async-await-explique-via-diagrammes-et-exemples/`.

1.8.2. Une introduction aux fonction asynchrones

La déclaration `async function` définit une fonction asynchrone qui renvoie un objet AsyncFunction. Une fonction asynchrone est une fonction qui s'exécute de façon asynchrone grâce à la boucle d'évènement en utilisant une promesse (Promise) comme valeur de retour.

```
async function trucAsynchrone(){
    console.log('calling');
    const result = await truc(); //On attend que la promesse soit résolue
    console.log(result);
}

trucAsynchrone(); //OK ou KO
```

1.8.3. L'API fetch

Nous allons utiliser un client asynchrone natif de JS : fetch (https://developer.mozilla.org/fr/docs/Web/API/Fetch_API/Using_Fetch).

Pour tester les exemples suivants vous devez avoir une clef d'API d'open weather map.

```
const openWeatherMapApiKey= 'b78d4845283d8c0855b25b4ea6671bd9';

const urlBaseWeather = 'https://api.openweathermap.org/data/2.5/';
const facetsWeather= `,fr&appid=${openWeatherMapApiKey}&units=metric&lang=fr`;


function myFecth1(city){
    const urlWeather = `${urlBaseWeather}weather?q=${city}${facetsWeather}`;
    fetch(urlWeather)
        .then(response => response.json())
```

```
        .then(data => console.log(data))
        .catch(err => console(err));
}

async function myFetch2 (city) {
    const urlWeather = `${urlBaseWeather}weather?q=${city}${facetsWeather}`;
    let response = await fetch(urlWeather);
    let data = await response.json();
    console.log(data);
}

myFecth1('paris');
myFetch2('paris');
```

2. Exercices

2.1. Exercices

Dans les exercices qui suivent, vous ne pouvez modifier l'HTML que pour inclure vos scripts.

2.1.1. Exercice 1

Dans ce premier exercice, le click sur une image change l'image de l'autre.

Figure 2.3. L'image du bas en remplacée par celle du haut



Dans ce premier exercice vous devrez :

1. Charger le script,
2. Attendre la fin du chargement de la page (événement `load`), si le script est chargé avant la fin,
3. Récupérer les images cliquables (querySelectorAll) et l'image de rendu (querySelector),
4. Munir les images d'un écouteur addEventListener sur le click, dans la callback lire et modifier l'attribut source (getAttribute, setAttribute).

2.1.2. Exercice 2

Un fenêtre modale dans une application web est une fenêtre qui prend tout l'écran et qui prend le contrôle. Une solution pourrait-être réalisée en pure CSS3 avec la pseudo classe `:target`. Nous allons, ici proposer une solution JavaScript et CSS.

Nous souhaitons qu'un clic sur le bouton affiche une fenêtre modale, un clic sur la croix ou sur la zone grisée la masque.

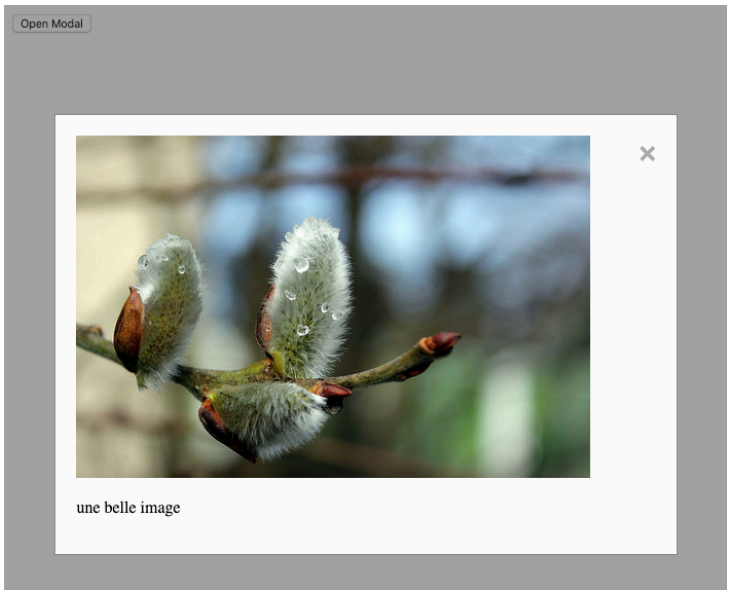
Vous disposer d'un fichier CSS avec la la règle :

```
.displayBlock{
    display: block;
}
```

Toute l'astuce est de rajouter ou de retirer cette classe à la modale. Vous devrez gérer l' événement "load" sur la fenêtre, l'événement "click" sur le bouton et le "span". L'attribut "classList" peut vous aider. Sur la body, vous devrez également gérer l'événement "click":

```
 document
        .querySelector('body')
        .addEventListener('click',  function(event) {
          if (event.target == modale) {
          //avec modale votre modale
}});
```

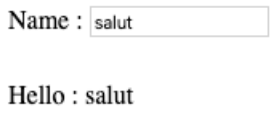
Figure 2.4. Modale visible



2.1.3. Exercice 3

Il est possible de stocker des informations dans votre navigateur pour la durée de la session (SessionStorage) ou pour une durée non définie (LocalStorage). L'objet windows dispose de 2 attributs pour réaliser ses opérations sessionStorage et localStorage. La page `exercice04-0.html` contient un `input`, sur entrée cette valeur est stockée dans le `sessionStorage` puis une redirection est faite vers `exercice04-1.html` qui récupère cette valeur.

Figure 2.5. sessionStorage



La redirection est réalisée avec `document.location.href` et pour reconnaître "`Enter`", il vous faudra vous renseigner sur `KeyboardEvent.code`.

2.1.4. Exercice 4

Vous trouverez ici, un site en react que nous allons partiellement reproduire sans framework: https://react-movie-database-18.netlify.app/.

2.1.4.1. Le DAO

Commencer par dans le module JS const.mjs par exporter votre API Key. Le module movieDAO contiendra votre DAO, ce dernier devra offrir trois méthodes :

- les films les plus populaires paramétrés par le numéro de la page (1 par défaut),

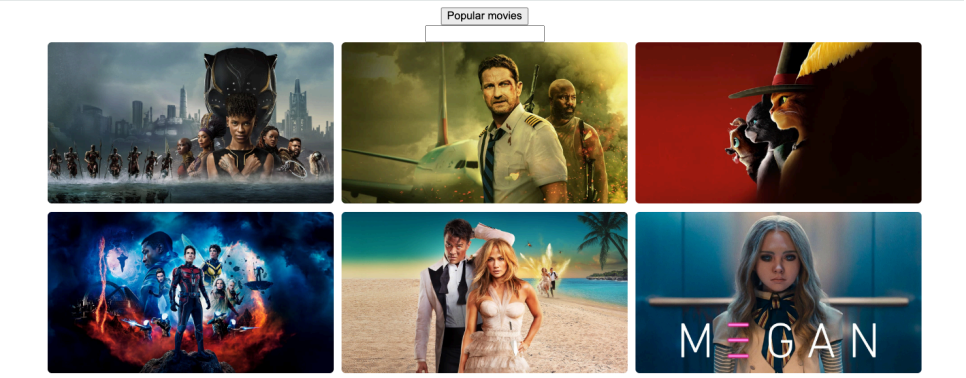
- un film connu par son id,
- une recherche par mot clef et par page (1 par défaut).

Vous pouvez retourner les littéraux bruts.

2.1.4.2. La page d'accueil

La page d'accueil permet d'afficher la première page des films populaires, le bouton "Popular movies" permet de la rafraîchir, le champ texte de faire une recherche. Le survol d'un film fait apparaître la date de publication, le titre et la note. Le click sur une image conduit à l'url suivante : movie.html?id=XXX avec XXX l'identifiant du film.

Figure 2.6. Page d'accueil (DOM)



Une CSS est fournie, pour que votre page rende bien, vous devrez respecter la structure suivante :

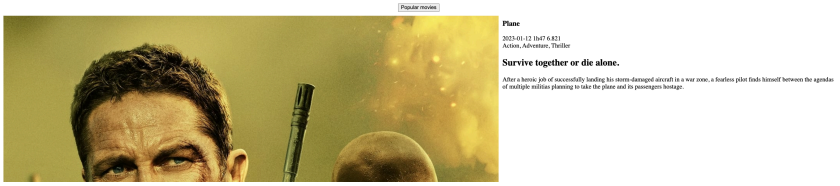
```
<article>
  <a href="movie.html?id=640146">
  <img alt="" src="https://image.tmdb.org/t/p/w1280/8YFL5QQVPy3AgrEQxNYVSgiPEbe.jpg">
  <div>
    <div>2023-02-10</div>
    <div>Ant-Man and the Wasp: Quantumania</div>
    <div>6.5</div>
  </div>
  </a>
</article>
```

L'événement keyup est utilisé sur le champ text.

2.1.4.3. La page film

La page film permet d'afficher de l'information sur le film sélectionné, le bouton permet de revenir sur la page d'accueil. Si la page est accédée sans id alors une redirection vers la page d'accueil est réalisée

Figure 2.7. film (DOM)



Pour parser l'url, tester la présence de l'id et récupérer sa valeur, vous pouvez utiliser window.location.search et un objet du type URLSearchParams.

2.1.4.4. L'analyse

Nous avons une application fonctionnelle mais qui souffre de plusieurs problèmes :

- Pas de modèle
- Difficulté à maintenir la cohérence
- Vue noyée dans le code et difficile à maintenir
- Code non ré utilisable
- Nombreux appel à l'API distante, par exemple lorsque l'on revient sur la page d'accueil.

Chapter 3. React

Nous allons découvrir React (React.js ou encore ReactJS) une bibliothèque JS utilisée par nombreux sites comme NetFlix, Yahoo, Instagram, ...

React n'est pas un framework mais repose sur les concepts suivants :

Composant

> Les composants vont nous offrir la modularité et la réutilisabilité, il communique en recevant des paramètres (props), dispose d'un état local et d'un cycle de vie

JSX (JavaScript Syntax Extension)

> Qui permet de structurer et simplifier le rendu des composants

Un DOM Virtuel

> Un DOM Virtuel est une représentation du DOM en JavaScript, nos script dialogue avec le DOM virtuel qui va optimiser les manipulations de la DOM.

La documentation de React se trouve ici : https://fr.reactjs.org/.

1. Composants

1.1. Cours

1.1.1. Un premier composant

Un composant en React est une association de HTML + CSS + JS. Les exemples suivants peuvent-être testé sur des plate-formes comme https://playcode.io/react ou https://codepen.io/ ou sur un de vos serveurs.

Un React les composants peuvent-être décrit sous forme de fonctions composant ou de classe composant, la première méthode est maintenant à privilégier.

Voici un premier exemple de code :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Add React in One Minute</title>
  </head>
  <body>
    <!-- We will put our React component inside this div. -->
    <div id="app_container"></div>

    <!-- Load React. -->
    <!-- Note: when deploying, replace "development.js" with "production.min.js". -->
    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>

    <!-- Load our React component. -->
    <script src="App.js"></script>

  </body>
</html>
```

Cette page contient, le conteneur DOM (`<div id="app_container"></div>`) qui sera géré par React et le chargement de la librairie React, du package de manipulation du DOM et votre script.

```
//App.js

class Hello extends React.Component {
  render() {
    return React.createElement('div', null, `Hello ${this.props.toWhat}`);
  }
}

const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(React.createElement(Hello, {toWhat: 'World'}, null));
```

Nous avons notre premier composant Hello dans lequel la méthode render a été redéfinie pour retourner une div contenant Hello suivi d'un paramètre fourni lors de la création (this.props.xxx). React ne gérera que la div d'identifiant '#app_container'. Cette div conteindra le composant Hello avec comme paramètre 'World'.

Dans la version fonction composant, les props sont reçues en paramètre et la méthode render remplacé par la valeur de retour.

```
const Hello= (props) => {
    return React
        .createElement('div', null, `Hello ${props.toWhat}`);

}
```

Le code peut encore être simplifié en utilisant l'opérateur de décomposition.

```
const Hello= ({toWhat}) => {
    return React
        .createElement('div', null, `Hello ${toWhat}`);

}
```

1.1.2. Ajout de JSX

Notre composant Hello est créé et réutilisable, cependant la syntaxe de createElement peut devenir lourde surtout avec des élément enfants (`createElement(type, props, ...children)`). JavaScript Syntax Extension permet de répondre a ce problème. JSX n'est plus du JS, nous devrons utiliser un système de traduction, ici Babel qui est utilisé à chaque consultation de la page.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Add React in One Minute</title>

    <script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
    <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>

  </head>
  <body>
    <!-- We will put our React component inside this div. -->
    <div id="app_container"></div>

    <!-- Load React. -->
    <!-- Note: when deploying, replace "development.js" with "production.min.js". -->

    <!-- Load our React component. -->
    <script type="text/babel" src="App.js"></script>

  </body>
</html>
```

```
class Hello extends React.Component {
  render() {
    return <div> Hello {this.props.toWhat}</div>
  }
```

```
}

const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<Hello toWhat={'world'}/>);
```

ou

```
function Hello({toWhat}) {
    return <div> Hello {toWhat}</div>
}

const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <div>
    <Hello toWhat={'world'}/>
        <Hello toWhat={'jojo'}/>
        <Hello toWhat={'en CM'}/>
    </div>)
```

1.1.3. Composons/décomposons nos composants

Un composant peut avoir des enfants et nous pouvons ainsi définir un arbre de dépendances. Le parent communiquant avec ses enfants en utilisant les props. Attention, les props sont en lecture seule, tout composant React doit agir comme une fonction pure vis-à-vis de ses props. Les composants doivent être les plus élémentaires possible.

Ce premier exemple montre la réutilisation de notre composant Hello appelé avec différents props :

```
root.render(
    <div>
    <Hello toWhat={'world'}/>
    <Hello toWhat={'à toi'}/>
    </div>
```

Il est annoter qu'il faut un composant racine ici une div.

Dans l'exemple qui suit nous allons décomposer un composant monolithique en sous composants :

```
const formatDate = (date) => date.toLocaleDateString();
const comment = {
    date: new Date(),
    text: 'I hope you enjoy learning React!',
    author: {
        name: 'Hello Kitty',
        avatarUrl: 'https://cataas.com/cat',
    },
};

class Comment extends React.Component {
    render() {
        return (
            <div className="Comment">
                <div className="UserInfo">
                    <img
                        className="Avatar"
                        src={this.props.author.avatarUrl}
                        alt={this.props.author.name}
                    />
                    <div className="UserInfo-name">
                        {this.props.author.name}
                    </div>
                </div>
                <div className="Comment-text">{this.props.text}</div>
                <div className="Comment-date">
                    {formatDate(this.props.date)}
                </div>
            </div>
        );
    }
```

```
}



const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <Comment
        date={comment.date}
        text={comment.text}
        author={comment.author}
    />
);
```

ou

```
const formatDate = (date) => date.toLocaleDateString();
const comment = {
    date: new Date(),
    text: 'I hope you enjoy learning React!',
    author: {
        name: 'Hello Kitty',
        avatarUrl: 'https://cataas.com/cat',
    },
};

const Comment = ({author,date,text})=>
            <div className="Comment">
                <div className="UserInfo">
                    <img
                        className="Avatar"
                        src={author.avatarUrl}
                        alt={author.name}
                    />
                    <div className="UserInfo-name">
                        {author.name}
                    </div>
                </div>
                <div className="Comment-text">{text}</div>
                <div className="Comment-date">
                    {formatDate(date)}
                </div>
            </div>




const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <Comment
        date={comment.date}
        text={comment.text}
        author={comment.author}
    />
);
```

Notre composant Comment peut-être décomposé comme suit :

```
class Avatar extends React.Component{
    render() {
        return (
            <img
                className="Avatar"
                src={this.props.user.avatarUrl}
                alt={this.props.user.name}
            />
        );
    }
}

class UserInfo extends React.Component{
```

```
    render() {
        return (
            <div className="UserInfo">
                <Avatar user={this.props.user}/>
                <div className="UserInfo-name">{this.props.user.name}</div>
            </div>
        );
    }
}


class Comment extends React.Component {
    render() {
        return (
            <div className="Comment">
                <UserInfo user={this.props.author} />
                <div className="Comment-text">{this.props.text}</div>
                <div className="Comment-date">
                    {formatDate(this.props.date)}
                </div>
            </div>
        );
    }
}
```

ou

```
const formatDate = (date) => date.toLocaleDateString();
const comment = {
    date: new Date(),
    text: 'I hope you enjoy learning React!',
    author: {
        name: 'Hello Kitty',
        avatarUrl: 'https://cataas.com/cat',
    },
};

const Avatar = ({user}) =>
            <img
                className="Avatar"
                src={user.avatarUrl}
                alt={user.name}
            />


const UserInfo  = ({user}) =>
            <div className="UserInfo">
                <Avatar user={user}/>
                <div className="UserInfo-name">{user.name}</div>
            </div>


const  Comment = ({author,text,date}) =>
            <div className="Comment">
                <UserInfo user={author} />
                <div className="Comment-text">{text}</div>
                <div className="Comment-date">
                    {formatDate(date)}
                </div>
            </div>



const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <Comment
        date={comment.date}
        text={comment.text}
        author={comment.author}
    />
```

```
);
```

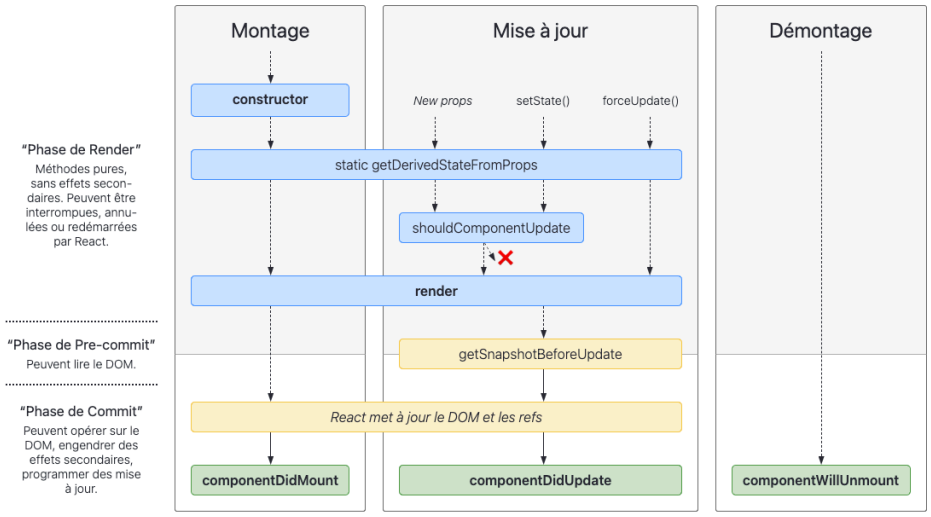
1.1.4. État local et cycle de vie

Les classes composant reçoivent des informations via les props mais peuvent également disposer d'un état interne `this.state`.

Les fonctions composant reçoivent des information via les paramètres mais peuvent également disposer d'un état interne accessible via le hook useState.

Les composants ont également un cycle de vie.

Figure 3.1. Cycle de vie d'un composant React



Nous allons l'illustrer avec un timer, selon les deux approches

1.1.4.1. Classe composant

```
const formatDate = (date) => date.toLocaleDateString();


class Clock extends React.Component {
    constructor(props) {
        super(props);
        this.state = {date: new Date()};
    }

    componentDidMount() {
        this.timerID = setInterval(
            () => this.tick(),
            1000
        );
    }

    componentWillUnmount() {
        clearInterval(this.timerID);
    }

    tick() {
        this.setState({
            date: new Date()
        });
    }

    render() {
```

```
        return (
            <div>
                <h1>Hello, world!</h1>
                <h2>It is {this.state.date.toLocaleTimeString()}.</h2>
            </div>
        );
    }
}



const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <Clock
    />
);

setInterval()
```

L'état local d'un composant est défini par this.state, il ne peut être modifié directement que dans le constructeur, sinon il faut passer par setState. Attention, React va optimiser de manière asynchrone les changements d'états aussi la valeur du prochain état ne peut faire directement référence à la valeur précédante.

```
// Erroné
this.setState({
  counter: this.state.counter + this.props.increment,
});

// Correct
this.setState((state, props) => ({
  counter: state.counter + props.increment
}));
```

De même les setState sont fusionnés, vous pouvez faire plusieurs setSate sur état composite :

```
//Avec comme state
//{
      posts: [],
      comments: []
    }
componentDidMount() {
    fetchPosts().then(response => {
      this.setState({
        posts: response.posts
      });
    });

    fetchComments().then(response => {
      this.setState({
        comments: response.comments
      });
    });
  }
```

La fusion n’est pas profonde, donc this.setState({comments}) laisse this.state.posts intacte, mais remplace complètement this.state.comments.

Analysons notre code :

1. Lors du render de root, le composant Clock est créé
2. Son état est défini dans le constructeur `this.state = {date: new Date()};`
3. Le render du Composant Clock est invoqué, qui va utiliser le state `{this.state.date.toLocaleTimeString()}`
4. Au moment de l'insertion dans la DOM du résultat du render la méthode componentDidMount() du cycle de vie est invoquée.

5. La méthode componentDidMount() met en place le minuteur `this.timerID = setInterval( () => this.tick(), 1000 )` qui invoke la méthode tick toutes les secondes.

6. La méthode `tick() { this.setState({ date: new Date() }); }` modifie l'état. L'état état étant modifié le render est de nouveau appelé.

7. Si le composant est retiré du DOM alors le timer est supprimé componentWillUnmount().

1.1.4.2. Fonctions composant

Les fonctions composante ne dispose pas de méthodes pour interagir avec le cycle de vie, React propose un mécanisme de hook pour y remédier. Les hook sont soumis à des contraintes, ils ne peuvent être appelés à l'intérieur de boucles, de code conditionnel ou de fonctions imbriquées, les hooks ne peuvent-être appelés que depuis des fonctions composant.

Un hook est une fonction qui permet de « se brancher » (to hook up) sur des fonctionnalités

React dispose de hook de base (useState, useEffect, useContext) de hook supplémentaires (useReducer useCallback useMemo useRef useImperativeHandle useLayoutEffect useDebugValue useDeferredValue useTransition useId), de hook de librairie (useSyncExternalStore, useInsertionEffect) et de hook spécialisés. Nous allons voir les deux premiers.

1.1.4.2.1. useState

Voici l'exemple de la documentation officielle (`https://legacy.reactjs.org/docs/hooks-state.html`) :

```
unction Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

count est la variable d'état, setCount le setter et 0 la valeur par initiale de count, dans cet exemple chaque click incrémente count.

1.1.4.2.2. useEffect

Ce hook est plus complexe, la documentation se trouve ici `https://legacy.reactjs.org/docs/hooks-reference.html#useeffect`.

useEffectpermet d'effectuer des effets : cela permet à notre composant d'exécuter des actions après l'affichage, en choisissant à quel moment cette action doit être exécutée.

Le hook useEffect est appelé après chaque rendu de votre composant. Il est possible de préciser quelle modification de donnée déclenche les effets exécutés dans useEffect, avec le tableau de dépendances.

Un tableau de dépendances vide permet d'exécuter un effet uniquement au premier rendu de votre composant.

Voici une correspondance approximative entre les méthodes du cycle de vie et useEffect

```
//componentDidMount
useEffect(() => {
  //code here
}, [])
```

```
//componentDidUpdate
useEffect(() => {
  //code here
}, [x,y,z]);

//where x,y,z are state variables on whose update,
//this method should get triggered

//componentDidUnmount
useEffect(() => {
  //code here
  return function() {
    //code to be run during unmount phase
  }
}, []);
```

Enfin, notre timer :

```
//spécifique à babel
const { useState, useEffect } = React
const formatDate = (date) => date.toLocaleDateString();

const Clock = () =>  {
    const [date,  setDate] = useState(new Date())
    const tick= () =>  {setDate(new Date())}
    const timer = (duration) => setInterval(()=>tick(),duration)

    const timerID = useEffect(() => {
        const timerID = timer(1000)
        return ()=> clearInterval(timerID)
    }, []);

   return (
            <div>
                <h1>Hello, world!</h1>
                <h2>It is {date.toLocaleTimeString()}.</h2>
            </div>)

}
```

1.1.5. Les événements

Les événements en React sont encapsulés dans SyntheticEvent mais leur utilisation reste sensiblement la même. Cependant le addEventListener ne peut-être utilisé qu'après que le composant soit monté aussi la déclaration de l'écouteur dans le composant est favorisée :

```
//Ecouteur sur le composant
class Toggle extends React.Component {
    constructor(props) {
        super(props);
        this.state = {isToggleOn: true};
    }

    render() {
        return (
            <button onClick={() => this.handleClick()}>
                {this.state.isToggleOn ? 'ON' : 'OFF'}
            </button>
        );
    }
    handleClick() {

        this.setState(state => ({
            isToggleOn: !state.isToggleOn
        }));
    }

}
```

```
const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(
    <Toggle
    />
);
```

ou

```
const { useState} = React
const Toggle = ()=> {
    const [isToggleOn, toggle] = useState(true)
     const   handleClick = ()=> {
        toggle(!isToggleOn)
     }

    return (
            <button onClick={() => handleClick()}>
                {isToggleOn ? 'ON' : 'OFF'}
            </button>
        );
}
```

Un zoom sur le this dans la fonction d'appel, dans le code précédant nous avons utilisé une fonction fléchée `() => handleClick()` lors de l'exécution de cette dernière le this est celui du contexte donc celui d'un objet de type Toggle. Sans la fonction fléchée le code suivant ne fonction pas car lors de la transformation en objet DOM le this de la classe ne sera plus défini.

```
<button onClick={this.handleClick}>
```

Pour retrouver le this de l'instance du composant, il faut réaliser un binding :

```
constructor(props) {
        super(props);
        this.state = {isToggleOn: true};
        this.handleClick=this.handleClick.bind(this);
    }
```

1.1.6. Les formulaires

En React, les formulaires peuvent être gérés. Le composant React qui affiche le formulaire contrôle aussi son comportement par rapport aux saisies de l'utilisateur. Un champ de formulaire dont l'état est contrôlé de cette façon par React est appelé un « composant contrôlé ».

```
class MyForm extends React.Component {
    constructor(props) {
        super(props);
        this.state = {
            text: ""
        };

        this.handleInputChange = this.handleInputChange.bind(this);
    }

    handleInputChange(event) {
        const target = event.target;
        const value = target.type === 'checkbox' ? target.checked : target.value;
        const type = target.type;

        this.setState({
            text : value + type
        });
    }

    render() {
        return (
```

```
            <form>
                <p> {this.state.text} </p>
                    <input
                        type="checkbox"
                        onChange={this.handleInputChange} />
                    <input
                        type="number"
                        onChange={this.handleInputChange} />

            </form>
        );
    }
}

const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<MyForm />);
```

Comme vu, précédemment, ce code contient une particularité, dans le onChange, nous utilisons this.handleInputChange or dans le navigateur le this ne sera plus la classe mais l'événement, le bind (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind) permet de continuer à rendre visible le this du composant React.

La version en fonction composant :

```
const { useState } = React
const MyForm = () =>  {
    const [text,  setText] = useState("")
        const handleInputChange= (event) =>{
        const target = event.target;
        const value = target.type === 'checkbox' ? target.checked : target.value;
        const type = target.type;
        setText(value +' '+ type);}

    return (

            <form>
                <p> {text} </p>
                    <input
                        type="checkbox"
                        onChange={handleInputChange} />
                    <input
                        type="number"
                        onChange={handleInputChange} />

            </form>
        );

}
```

1.1.7. Les affichages conditionnels

Un premier exemple avec un composant sélectionné en fonction de la valeur d'une props :

```
"use strict"
class UserGreeting extends React.Component {
    constructor(props) {
        super(props);
    }
    render() {
        return (
            <h1>Bienvenue !</h1>
        );
    }
}

class GuestGreeting extends React.Component {
    constructor(props) {
        super(props);
    }
```

```
    render() {
        return (
            <h1>Veuillez vous inscrire.</h1>
    );
    }
}

class Greeting extends React.Component {
    constructor(props) {
        super(props);
    }
    render() {
        if (this.props.isLoggedIn) {
            return <UserGreeting />;
        }
        return <GuestGreeting />;
    }
}

const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<Greeting isLoggedIn={false}/>);
```

```
const { useState } = React
const UserGreeting = () => <h1>Bienvenue !</h1>
const GuestGreeting = ()=> <h1>Veuillez vous inscrire.</h1>


const Greeting = ({isLoggedIn}) =>
         (isLoggedIn) ?  <UserGreeting /> : <GuestGreeting />
```

Un deuxième exemple avec les éléments dans une variable `{button}`.

```
class LoginButton extends React.Component {
    constructor(props) {
        super(props);
    }
    render() {
        return (
            <button onClick={this.props.onClick}>
                Connexion
            </button>
        );
    }
}

class LogoutButton extends React.Component {
    constructor(props) {
        super(props);
    }
    render() {
        return (
            <button onClick={this.props.onClick}>
                Déconnexion
            </button>
        );
    }
}

class LoginControl extends React.Component {
    constructor(props) {
        super(props);
        this.handleLoginClick = this.handleLoginClick.bind(this);
        this.handleLogoutClick = this.handleLogoutClick.bind(this);
        this.state = {isLoggedIn: false};
    }

    handleLoginClick() {
        this.setState({isLoggedIn: true});
    }
```

```
    handleLogoutClick() {
        this.setState({isLoggedIn: false});
    }

    render() {
        const isLoggedIn = this.state.isLoggedIn;
        let button;
        if (isLoggedIn) {
            button = <LogoutButton onClick={this.handleLogoutClick} />;
        } else {
            button = <LoginButton onClick={this.handleLoginClick} />;
        }

        return (
            <div>
                <Greeting isLoggedIn={isLoggedIn} />
                {button}
            </div>
        );
    }
}
```

La version fonction composant :

```
const { useState } = React
const UserGreeting = () => <h1>Bienvenue !</h1>
const GuestGreeting  = () => <h1>Veuillez vous inscrire.</h1>
const Greeting = ({isLoggedIn}) =>
        isLoggedIn ? <UserGreeting />: <GuestGreeting />

const LoginButton=({onClick}) =>
            <button onClick={onClick}>
                Connexion
            </button>

const LogoutButton = ({onClick}) =>
                <button onClick={onClick}>
                Déconnexion
            </button>

const LoginControl = () => {
    const [isLoggedIn, toggle] = useState(false)
    const handleLoginClick= () => {
        toggle(true);
    }
    const handleLogoutClick = () => {
        toggle(false);
    }
    const button = (isLoggedIn) ?
            <LogoutButton onClick={handleLogoutClick} /> :
            <LoginButton onClick={handleLoginClick} />;


    return (
            <div>
                <Greeting isLoggedIn={isLoggedIn} />
                {button}
            </div>
        );

}
```

Le && et l'opérateur ternaire permettent des affichages conditionnels, de même qu'un null empêchera un affichage :

```
"use strict"
class WarningBanner extends React.Component {
    constructor(props) {
        super(props);
    }
```

```
    render() {
        if (!this.props.warn) {
            return null;
        }

        return (
            <div className="warning">
                Attention !
            </div>
        );
    }
}

class Page extends React.Component {
    constructor(props) {
        super(props);
        this.state = {showWarning: true};
        this.handleToggleClick = this.handleToggleClick.bind(this);
    }

    handleToggleClick() {
        this.setState(state => ({
            showWarning: !state.showWarning
        }));
    }

    render() {
        return (
            <div>
                <WarningBanner warn={this.state.showWarning} />
                <button onClick={this.handleToggleClick}>
                    {this.state.showWarning ? 'Masquer' : 'Afficher'}
                </button>
            </div>
        );
    }
}
const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<Page />);
```

```
const { useState} = React

const WarningBanner = ({warn})=>
    (!warn) ? null :
           <div className="warning">
                Attention !
           </div>


const Page = () =>  {
    const [showWarning, toggle] = useState(true)
    const handleToggleClick = () => {
        toggle(!showWarning)
    }
    return (
           <div>
                <WarningBanner warn={showWarning} />
                <button onClick={handleToggleClick}>
                    {showWarning ? 'Masquer' : 'Afficher'}
                </button>
            </div>
        );
}
```

1.1.8. Les listes

Les éléments de liste doivent avoir une clef (key) pour permettre à React de gérer les modifications :

```
"use strict"
```

```
class Blog extends React.Component {
    constructor(props) {
        super(props);
    }

    render() {
        const sidebar = (
            <ul>
                {this.props.posts.map((post) =>
                    <li key={post.id}>
                        {post.title}
                    </li>
                )}
            </ul>
        )
        const content = this.props.posts.map((post) =>
            <div key={post.id}>
                <h3>{post.title}</h3>
                <p>{post.content}</p>
            </div>
        );
        return (
            <div>
                {sidebar}
                <hr/>
                {content}
            </div>
        );
    }
}
const posts = [
    {id: 1, title: 'Bonjour, monde', content: 'Bienvenue sur la doc de React !'},
    {id: 2, title: 'Installation', content: 'Vous pouvez installer React depuis npm.'}
];
const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<Blog posts={posts} />);
```

La version fonction composant :

```
"use strict"
const Blog = ({posts}) => {
        const sidebar = (
            <ul>
                {posts.map((post) =>
                    <li key={post.id}>
                        {post.title}
                    </li>
                )}
            </ul>
        )
        const content = posts.map((post) =>
            <div key={post.id}>
                <h3>{post.title}</h3>
                <p>{post.content}</p>
            </div>
        );
        return (
            <div>
                {sidebar}
                <hr/>
                {content}
            </div>
        );
}
const posts = [
    {id: 1, title: 'Bonjour, monde', content: 'Bienvenue sur la doc de React !'},
    {id: 2, title: 'Installation', content: 'Vous pouvez installer React depuis npm.'}
];
```

1.1.9. Faire remonter un état

Pour le moment, nous ne savons que communiquer entre composants imbriqués, un composant père peut communiquer avec ses enfants au travers des props, il peut grâce à ce mécanisme passer des traitements sur son état et ainsi permettre aux enfants de partager un état commum.

```
"use strict"

const scaleNames = {
    c: 'Celsius',
    f: 'Fahrenheit'
};

function toCelsius(fahrenheit) {
    return (fahrenheit - 32) * 5 / 9;
}

function toFahrenheit(celsius) {
    return (celsius * 9 / 5) + 32;
}

function tryConvert(temperature, convert) {
    const input = parseFloat(temperature);
    if (Number.isNaN(input)) {
        return '';
    }
    const output = convert(input);
    const rounded = Math.round(output * 1000) / 1000;
    return rounded.toString();
}

class BoilingVerdict extends React.Component {
    constructor(props) {
        super(props);
    }
    render () {
        if (this.props.celsius >= 100) {
            return <p>The water would boil.</p>;
        }
        return <p>The water would not boil.</p>;
    }
}

class TemperatureInput extends React.Component {
    constructor(props) {
        super(props);
        this.handleChange = this.handleChange.bind(this);
    }

    handleChange(e) {
        this.props.onTemperatureChange(e.target.value);
    }

    render() {
        const temperature = this.props.temperature;
        const scale = this.props.scale;
        return (
            <fieldset>
                <legend>Enter temperature in {scaleNames[scale]}:</legend>
                <input value={temperature}
                       onChange={this.handleChange} />
            </fieldset>
        );
    }
}

class Calculator extends React.Component {
    constructor(props) {
        super(props);
```

```
        this.handleCelsiusChange = this.handleCelsiusChange.bind(this);
        this.handleFahrenheitChange = this.handleFahrenheitChange.bind(this);
        this.state = {temperature: '', scale: 'c'};
    }

    handleCelsiusChange(temperature) {
        this.setState({scale: 'c', temperature});
    }

    handleFahrenheitChange(temperature) {
        this.setState({scale: 'f', temperature});
    }

    render() {
        const scale = this.state.scale;
        const temperature = this.state.temperature;
        const celsius = scale === 'f' ? tryConvert(temperature, toCelsius) : temperature;
        const fahrenheit = scale === 'c' ? tryConvert(temperature, toFahrenheit) : temperature;

        return (
            <div>
                <TemperatureInput
                    scale="c"
                    temperature={celsius}
                    onTemperatureChange={this.handleCelsiusChange} />
                <TemperatureInput
                    scale="f"
                    temperature={fahrenheit}
                    onTemperatureChange={this.handleFahrenheitChange} />
                <BoilingVerdict
                    celsius={parseFloat(celsius)} />
            </div>
        );
    }
}
const root = ReactDOM.createRoot(document.querySelector('#app_container'));
root.render(<Calculator />);

>);
```

Le Calculator maintient la température et l'état de la conversion `{temperature: '', scale: 'c'}`, il contiend deux `TemperatureInput`. Lors de d'un changement l'input d'un TemperatureInput, la méthode appelée est celle du Calculator (`this.props.onTemperatureChange(e.target.value)` est associée à `this.handleFahrenheitChange` ou `this.handleCelsiusChange` qui met à jour l'état du Calculator (this.setSate()), le rendeur est appelé et les composants imbriqués se mettent à jour.

La version fonction composant :

```
const BoilingVerdict = ({celsius}) =>
        (celsius >= 100)?
            <p>The water would boil.</p>
        : <p>The water would not boil.</p>


const TemperatureInput=({onTemperatureChange,temperature,scale}) =>{
    const handleChange=(e) => onTemperatureChange(e.target.value);
    return (
            <fieldset>
                <legend>Enter temperature in {scaleNames[scale]}:</legend>
                <input value={temperature}
                       onChange={handleChange} />
            </fieldset>
        )
}

const Calculator = ()=>  {
        const [temperature,setTemperature]=useState('')
        const [scale, setScale]=useState('c')
```

```
    const handleCelsiusChange = (temperature) => {
        setScale('c')
        setTemperature(temperature)
    }

    const handleFahrenheitChange=(temperature) =>{
        setScale('f')
        setTemperature(temperature)
    }

    const celsius = scale === 'f' ? tryConvert(temperature, toCelsius) : temperature
    const fahrenheit = scale === 'c' ? tryConvert(temperature, toFahrenheit) : temperature;

        return (
            <div>
                <TemperatureInput
                    scale="c"
                    temperature={celsius}
                    onTemperatureChange={handleCelsiusChange} />
                <TemperatureInput
                    scale="f"
                    temperature={fahrenheit}
                    onTemperatureChange={handleFahrenheitChange} />
                <BoilingVerdict
                    celsius={parseFloat(celsius)} />
            </div>
        );

}
```

1.2. Exercices

1.2.1. Le tutoriel officiel

Commençons par les tutoriaux officiels : `https://react.dev/learn`, `https://react.dev/learn/tutorial-tic-tac-toe`. Nous allons maintenant reprendre les exercices de la première partie.

1.2.2. Exercice 1

Nous allons reproduire l'exercice sur les tuxsSection 2.1.1, “Exercice 1”. L'exercice est a reproduire avec trois composants :

- ImageCible qui affichera l'image sélectionnée
- ImageClickable qui permet de sélectionner l'image
- App qui contiendra les autres composants, l'état de App sera composé de l'image sélectionnée.

Vous pouvez vous inspirer de l'exemple qui fait remonter l'état.

1.2.3. Exercice 2

Nous allons reprendre l'exercice sur la modale de la partie DOMSection 2.1.2, “Exercice 2”. Ici, l'état est défini par la présence de la classe displayBlock sur la div modale. Je vous conseille trop composants : le bouton, la modale et l'application. La partie click sur le body ne pourra être gérée.

1.2.4. Exercice 3

Nous allons reprendre l'exercice sur les films de la partie DOMSection 2.1.4, “Exercice 4”. Les imports ne pouvant fonctionner avec babel en mode stand-alone, le DAO devra être copié dans les scripts react.

Le coeur de notre modèle sera :

```
class App extends React.Component {
```

```
    constructor(props) {
        super(props)
        this.state = {
            query : "",
            movies : []
        }
        this.doUpdate = this.doUpdate.bind(this)
    }
    componentDidMount() {
        this.doUpdate()
    }

    doUpdate(query){
        if (query != undefined)
            this.setState({query: query})

        if (this.state.query == "")
            movieDAO.getPopulars()
                .then(data => {
                    this.setState({movies: data.results})
                })
        else
            movieDAO.find(this.state.query)
                .then(data => {
                    this.setState({movies: data.results})
                })
    }
...
```

2. Composants communicants et routage (non évalué en cours)

Dans l'exemple de notre site, nous avons toujours deux pages et une communication uniquement entre composants d'une même hiérarchie, une impossibilité d'imports multiples, nous allons remédier à ces problèmes.

2.1. Un CLI

Lors de l'utilisation de frammework, les Command Ligne Interface permettent de faciliter les développement, pour React les CLI vont apporter :

Un compilateur

: Qui va nous permettre de compiler nos composant JSX et CSS pour nous se sera babel

Un bundler

: Qui permet d'assembler, compresser le code, pour ce sera webpack

Un gestionnaire de paquet

: npm ou yarn pour nous

Vous trouverez ici la documentation du CLI officiel de reactJS : `https://fr.legacy.reactjs.org/docs/create-a-new-react-app.html#create-react-app`.

1. Créer une première application
2. Lancer l'application avec le serveur de test
3. Modifier le message "Edit src/App.js and save to reload."
4. Produire une version de production et tester avec un serveur local
5. Recommander pour une publication non à la racine du serveur (Building for Relative Paths)

2.2. Le routage

Pour obtenir un Single Page Application, il nous faut un routeur dont le rôle est en fonction de l'URL de changer le contenu de la DOM en restant sur la même page. Vous trouverez ici la documentation officielle : `https://reactrouter.com/en/main`.

A vous de reproduire le comportement suivant :

/

La page d'accueil avec deux liens movie/1, mouvie/2

/movie/:movieID

Une page qui affiche l'id reçu et propose un lien vers la page d'accueil

Les urls non gérées devront conduire à une page d'erreur avec un lien vers la page d'accueil.

Le tutoriel est intéressant mais un peu long, vous devriez pouvoir vous en sortir en regardant la documentation de BrowserRouter, Routes, Route, Link, useParams.

2.3. Propager les props

Les contextes sont le moyen le plus simple de partager un état et d'éviter la propagation des props, le code suivant illustre ce problème :

```
class App extends React.Component {
  render() {
    return <Toolbar theme="dark" />;
  }
}

function Toolbar(props) {
  // Le composant Toolbar doit prendre une prop supplémentaire `theme` et la
  // passer au ThemedButton. Ça peut devenir pénible si chaque bouton de l'appli
  // a besoin de connaître le thème parce qu'il faudra le faire passer à travers
  // tous les composants.
  return (
    <div>
      <ThemedButton theme={props.theme} />
    </div>
  );
}

class ThemedButton extends React.Component {
  render() {
    return <Button theme={this.props.theme} />;
  }
}
```

En utilisant un contexte ont peut passer des props implicitement au travers d'une hiérarchie :

```
// Le Contexte nous permet de transmettre une prop profondément dans l'arbre des
// composants sans la faire passer explicitement à travers tous les composants.
// Crée un contexte pour le thème (avec "light" comme valeur par défaut).
const ThemeContext = React.createContext('light');

class App extends React.Component {
  render() {
    // Utilise un Provider pour passer le thème plus bas dans l'arbre.
    // N'importe quel composant peut le lire, quelle que soit sa profondeur.
    // Dans cet exemple, nous passons "dark" comme valeur actuelle.
    return (
      <ThemeContext.Provider value="dark">
        <Toolbar />
      </ThemeContext.Provider>
    );
```

```
  }
}

// Un composant au milieu n'a plus à transmettre explicitement le thème
function Toolbar() {
  return (
    <div>
      <ThemedButton />
    </div>
  );
}

class ThemedButton extends React.Component {
  // Définit un contextType pour lire le contexte de thème actuel.  React va
  // trouver le Provider de thème ancêtre le plus proche et utiliser sa valeur.
  // Dans cet exemple, le thème actuel est "dark".
  static contextType = ThemeContext;
  render() {
    return <Button theme={this.context} />;
  }
}
```

Réaliser le tutorial suivant en prenant soin de bien comprendre les notions de provider et consummer : `https://www.geeksforgeeks.org/explain-new-context-api-in-react/`

L'approche contexte à un inconvénient, tous les composants consommant le contexte vont être re-render à chaque changement du state global proposé par le contexte, même si l'information qu'ils utilisent n'est pas changée. Les gestionnaires d'état répondent à ce problème, Redux est le plus populaire et permet de gérer plus finement les changement d'états.

2.4. Redux

La documentation officielle se trouve ici : `https://redux.js.org/`.

Dans Redux, l'état global est le "Sate", il n'est pas modifié directement mais grâce à des actions. Une action est objet JS qui décrit la raison du changement de l'état et sa valeur grâce à la propriété type et au payload. Si les actions sont produites par des fonctions, on parle d'"Action Creator".

```
//Action créator
const playPause = () => ({ type: "playPause" });

const pointScored = (player) => ({
    type: "pointScored",
    payload: { player: player },
});
```

La gestion des actions et le maintient de l'état et fait avec les "Reducer" des fonctions qui reçoivent le state et une action en paramètre, et qui retourne un nouveau state.

```
// le reducer est une fonction
function reducer(state, action) {
    // si l'action est de type playPause...
    if (action.type === "playPause") {
        // ... il faut inverser la propriété playing du state
        return {
            ...state, //Important
            playing: !state.playing
        };
    }
    // sinon on retourne le state sans le changer
    return state;
}
```

Il est a noter l'opérateur de destructuration (...) qui permet de créer un nouveau "state" sans modifier le "state" précédent.

Le store fait le lien avec les actions, les reducers et les actions :

```
import { createStore } from "https://cdn.skypack.dev/redux@4.0.5";
```

Le State est créé à partir du state et du reducer

```
// on crée le store avec le state et le reducer
const store = createStore(reducer, initialState);
```

L'accès au state se fait avec getState()

```
const state = store.getState();
// { playing: true, ... }
```

La modification du "State", se fait via dispatch :

```
// on passe un objet action
store.dispatch({ type: "playPause" });
// on passe une action depuis une variable
store.dispatch(playPauseAction);
// on passe une action en utilisant un action creator
// attention à bien appeler la fonction pointScored !
store.dispatch(pointScored("player2"));
```

Pour finir, le React n'écoute pas le "State" de redux, la solution est d'appeler une méthode lors de la modification du state :

```
store.subscribe(
    // cette fonction sera exécutée à chaque fois que le state change
    () => {
        const state = store.getState();
        updateScoreText(state.playing);
    }
);
```

Les appel asynchrones ne seront pas gérés dans le store mais dans des fonctions externes qui vont lui demander des modifications.

Réaliser le tutoriel redux : `https://openclassrooms.com/fr/courses/8228131-gerez-votre-application-react-avec-redux-et-redux-toolkit`.